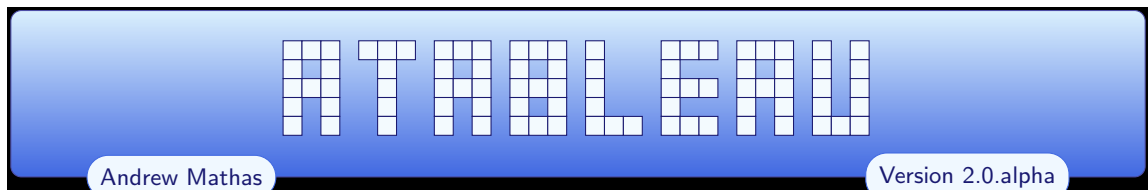


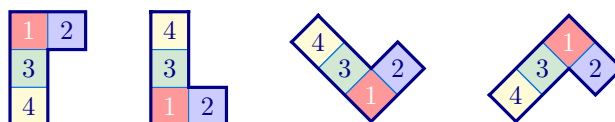


This version of the package is already quite powerful, but it is still in the proof-of-concept phase. There are still bugs and some commands are not yet documented. Several planned features have not been implemented. Everything is subject to change. Comments and suggestions are welcome.



CONTENTS

1. Introduction	1
2. Tableaux and Young diagrams	6
2A. Tableaux	6
2A.1. Adding style	7
2A.2. Tableau coordinates	9
2A.3. Tableau options	9
2A.4. Compositions	15
2A.5. Drawing order	15
2A.6. Special characters	15
2B. Young diagrams	16
2C. Skew tableaux and skew diagrams	18
2D. Shifted tableau and shifted diagrams	19
2E. Tabloids	20
2F. Ribbon tableaux	20
2F.1. Ribbon specifications	21
2G. Multidiagrams and multitableaux	23
3. Abacuses	25
4. Dynkin quivers?	26
5. Braid diagrams and KLRW diagrams?	26
6. Colours	26
7. Feature requests and bug reports	26
Index	27



1. INTRODUCTION

The package provides commands for drawing common objects in algebraic combinatorics and representation theory, together with styling options and a key-value interface for additional customization. Under the hood, everything is drawn using **TikZ**. These commands can be used either as standalone functions or as part of a **tikzpicture** environment. To use the package, add the following line to your document preamble:

```
\usepackage[options]{atableau}
```

Here, the *options* is an optional comma-separated list of **aTableau** package options — we describe the possible options below. For example, to make the tableaux and Young diagrams commands use the French convention, by default, you could use:

```
\usepackage[french]{atableau}
```

You can change the default options at any point in your document using the **\aTabset** command. Options can also be used with individual commands.

The [aTableau](#) package provides the following commands:

Command	Diagram
<code>\Abacus</code>	An abacus for a partition
<code>\Diagram</code>	A Young diagram for a partition
<code>\Multidiagram</code>	An ℓ -tuple of Young diagrams
<code>\Multitableau</code>	An ℓ -tuple of tableaux
<code>\RibbonTableau</code>	A ribbon tableau
<code>\ShiftedDiagram</code>	A shifted Young diagram
<code>\ShiftedTableau</code>	A shifted tableau
<code>\SkewDiagram</code>	A skew Young diagram
<code>\SkewTableau</code>	A skew tableau
<code>\Tableau</code>	A tableau
<code>\Tabloid</code>	A tabloid

The commands are designed to be easy to use, easy read, and are highly customisable, allowing for the different components of these diagrams. Each command accepts key-value options for modifying the diagrams and optional styling can be applied to their individual components. Each command can be used as a standalone object in a document, or they can be incorporated into a larger [tikzpicture](#) environment. This makes it possible to use these commands as part of more complicated diagrams.

The general syntax of an [aTableau](#) command is:

`\Command (x,y) [options] {...specifications...}`

The (x,y) -Cartesian coordinates are optional, being necessary if and only if the diagram is part of a [tikzpicture](#) environment. Similarly, the key-value *options* control the style and appearance of the final diagram.

The commands are intended to be easy to use. For example, partitions can be entered either as a (weakly decreasing list of non-negative) integers, or using exponential notation, or as a sequence of beta numbers (for the `\Abacus` command), and tableaux are entered as comma separated words. The commands support the common conventions. For example, the following table shows how to draw a Young diagram of shape $\lambda = (4, 3^2, 2)$ using the four tableaux conventions supported by [aTableau](#):

Convention	aTableau command	Diagram
English	<code>\Tableau[english]{1234,567,89{10},{11}{12}}</code> (the default)	
French	<code>\Tableau[french]{1234,567,89{10},{11}{12}}</code>	
Ukrainian ¹	<code>\Tableau[ukrainian]{1234,567,89{10},{11}{12}}</code>	
Australian ²	<code>\Tableau[australian]{1234,567,89{10},{11}{12}}</code>	

¹Some authors call this the Russian convention.

²I have not seen a name for this convention in the literature. Calling this the *Australian convention* seems apt.

This table shows how the commands provided by `aTableau` work: there are basic commands for drawing diagrams, and these diagrams can be embellished using optional arguments, which use a key-value interface.

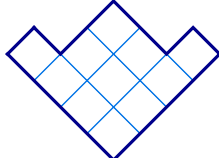
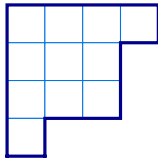
The different conventions for diagrams, and tableaux, are listed in order of (my perception of) their popularity in the literature. By default, the *English* convention is used, so we can omit *english*:

1	2	3	4
5	6	7	
8	9	10	
11	12		

```
\Tableau{1234,567,89{10},{11}{12}}
```

1.1

for the first tableau in the table. These four conventions can be used in exactly the same way with the `\Diagram` command:



```
\Diagram{4,3,3,1}
\Diagram[ukrainian]{4,3^2,1}
```

1.2

This example shows that partitions can either be typed out in full, as a comma-separated list, or using exponential notation, which is usually shorter and easier to read.

The possible options, or keys, for the `aTableau` commands are described in detail below, with examples. This section emphasises only some of these options together with the fact that all of these keys can be set “globally” (inside the current $\text{\LaTeX}$ group), by using `\aTabset{...options...}`. As discussed already, the default options for the document can also be set via `\usepackage[options]{atableau}`.

Many of the `aTableau` options are specific to the particular diagrams being drawn. All of the options are described in detail in later sections. The following options are common to all of the commands.

- align** : Sets the vertical alignment of the diagrams. The possible options are `align=top`, `align=center` and `align=bottom`, which aligns the baseline of the diagram with the top, centre or bottom of the surrounding objects, respectively. By default, `aTableau` are centered.
- name** : Set the prefix for the `TikZ` named anchors for the boxes and beads in `aTableau` diagrams.
- scale, xscale, yscale** : Use `scale` to rescale all `aTableau` diagrams. Use `xscale` to rescale in the x -direction and `yscale` to rescale in the y -direction.
- star style** : Adding a `*`-prefix to elements of an `aTableau` diagram is a shorthand for changing the style of that element. Use `star style` to change the default `*`-styling.
- tikzpicture** : All `aTableau` commands are drawn inside a `tikzpicture` environment. Use `tikzpicture=keys` to set the optional argument of this environment:

```
\begin{tikzpicture}[ keys ]...\end{tikzpicture}
```
- tikz before, tikz after** : Use the `tikz before` and `tikz after` keys to inject `TikZ` code into the `tikzpicture` environment, before and after the diagram drawn by `aTableau`.

For boolean options, which are set to `true` or `false`, the value can typically be omitted. For example, `border` is the same as `border=true` (and `no border` is the same as `border=false`). In all of the options, American spelling variations, such as *center* (instead of *centre*), *color* (instead of *colour*) et cetera, are tolerated. The names of the default colours used by `aTableau` are given in [Chapter 6](#).

The next example shows how, how to use the `\Diagram` command inside a `tikzpicture` environment by supplying Cartesian coordinates.

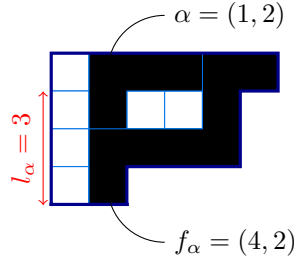
```
% \usetikzlibrary{patterns}
\begin{tikzpicture}[
```

1.3

```

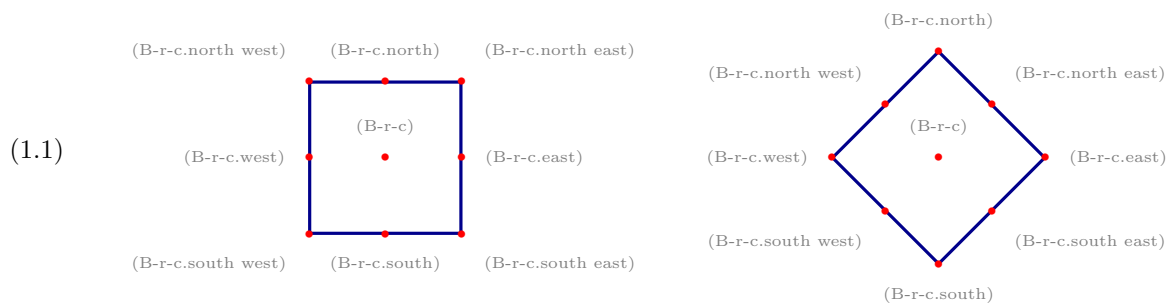
B/.style={pattern=north east lines, pattern color=blue,opacity=0.5},
G/.style={pattern=north west lines, pattern color=ForestGreen,opacity=0.5},
]
\Diagram(0,0)[ribbons={{(G)16ccccrrr, (B)16crrcccr}}]{6,5^2,2}
\draw[->](1.5, 0.5)node[right]{$\alpha=(1,2)$} to [out=180,in=90] (B-1-2.base);
\draw[->](1.5,-2.5)node[right]{$f_\alpha=(4,2)$} to [out=180,in=270] (B-4-2.base);
\draw[red,<->]([xshift=-1mm]B-1-1.south west)
--node[rotate=90,above]{$l_\alpha=3$}([xshift=-1mm]B-4-1.south west);
\end{tikzpicture}

```



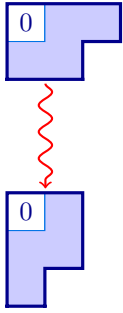
Most of the commands in the last example are **TikZ** commands, but most of the picture is drawn by the `\Diagram` command, with the `ribbons` key specifying two *ribbons*, which are the shaded boxes giving the $(1,2)$ -hook and $(1,2)$ -rim hook, respectively. The top of the example defines two **TikZ**-styles, **B** and **G**, which are the styles used by the two ribbons to shade the hooks. The Young diagram is placed at position $(0,0)$ by the `\Diagram` command. The three `\draw` commands are used to add the three arrows to the picture. Please consult the very readable **TikZ** manual if you would like more details on what the **TikZ**-commands are doing in this example.

From the viewpoint of **aTableau**, the most important point of the last example is that the `\Diagram` command can be used inside a `tikzpicture` environment because it is equipped with the Cartesian coordinate $(0,0)$. The example uses the four named coordinates: $(B-1-1)$, $(B-1-2)$, $(B-4-1)$, and $(B-4-2)$. When **aTableau** draws a tableau, or a diagram, it creates **TikZ** named coordinates for each of the boxes in the diagram, so that $(B-r-c)$ refers to the box in row r and column c . Giving finer control, the following standard **TikZ** anchors are available:



In the name $(B-r-c)$, r and c should be replaced by the row and column index of a box in the tableau, because these anchors are *only* defined for boxes in the tableau. These *anchors* are a standard part of **TikZ**. If you change the shape of the node, then the available anchors may change; for more information see the **TikZ** manual. You can change the **B** in the name of these anchors to anything you like using the `name` option. Similar anchors are created for abacus beads.

In the example above the Cartesian coordinate $(0,0)$ is necessary because the `\Diagram` command is used inside a `tikzpicture` environment, but the choice of coordinate is not important in the sense that changing the coordinate will not change the picture. Cartesian coordinates become more meaningful when the `tikzpicture` environment has more components. In the next example, notice the use of `name=A` to make the named coordinates for the first diagram take the form $(A-r-c)$, instead of the default of $(B-r-c)$.



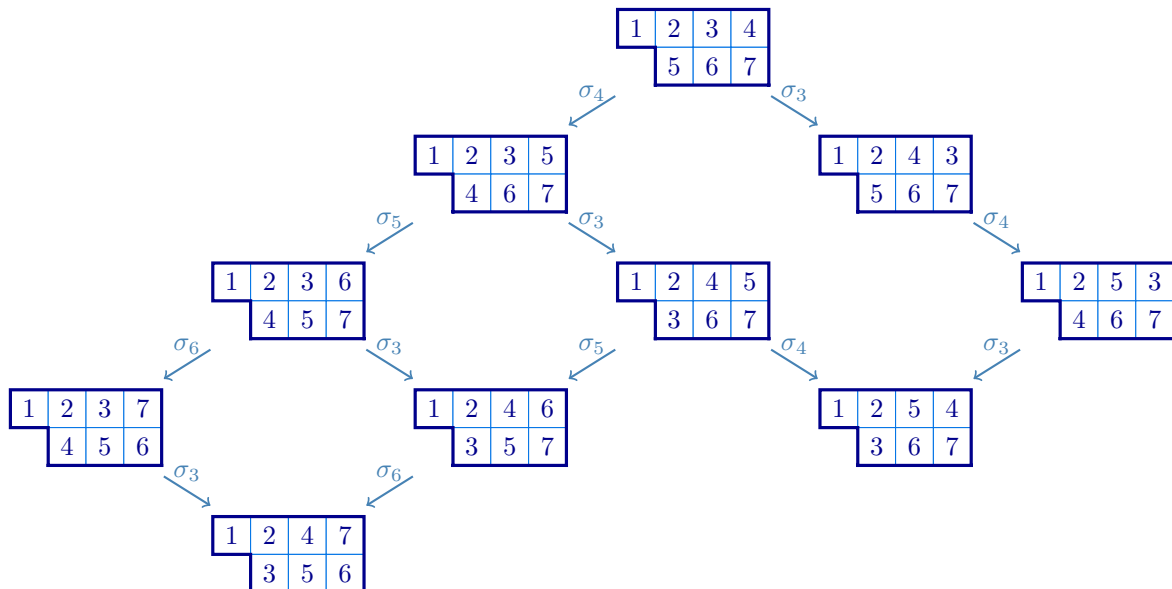
```
% \usetikzlibrary{decorations.pathmorphing}
\begin{tikzpicture}[wave/.style={thick,red,->,decorate,decoration=snake}]
  \aTabset{ribbon style={fill=blue!20, opacity=0.4}}
  \Diagram(0,0) [e=2,show=residues,ribbons={13crc},name=A]{3,2}
  \Diagram(0,-2.5) [e=2,show=residues,ribbons={12rcr}]{2^2,1}
  \draw[wave] ([shift={(0,-0.05)}]A-2-1.south east)
    --([shift={(0,0.05)}]B-1-1.north east);
\end{tikzpicture}
```

1.4

The final example in the introduction uses `\ShiftedTableau` inside a *matrix of nodes*:

```
% \usetikzlibrary{matrix}
\begin{tikzpicture}[scale=0.8, arr/.style={->,SteelBlue, thick}]
  \matrix (M)[matrix of nodes,row sep=4mm,column sep=4mm]{
    & & & \ShiftedTableau{1234,567} & \backslash \\
    & & \ShiftedTableau{1235,467} & & \\
    & & \ShiftedTableau{1243,567} & \backslash \\
    & \ShiftedTableau{1236,457} & & & \\
    & & \ShiftedTableau{1245,367} & & \\
    & & \ShiftedTableau{1253,467} & \backslash \\
    \ShiftedTableau{1237,456} & & & & \\
    & & \ShiftedTableau{1246,357} & & \\
    & & \ShiftedTableau{1254,367} & \backslash \\
    & \ShiftedTableau{1247,356} & \backslash \\
  };
  \draw[arr] (M-1-4) --node[above]{$\sigma_4$} (M-2-3);
  \draw[arr] (M-1-4) --node[above]{$\sigma_3$} (M-2-5);
  \draw[arr] (M-2-3) --node[above]{$\sigma_5$} (M-3-2);
  \draw[arr] (M-2-3) --node[above]{$\sigma_3$} (M-3-4);
  \draw[arr] (M-2-5) --node[above]{$\sigma_4$} (M-3-6);
  \draw[arr] (M-3-2) --node[above]{$\sigma_6$} (M-4-1);
  \draw[arr] (M-3-2) --node[above]{$\sigma_3$} (M-4-3);
  \draw[arr] (M-3-4) --node[above]{$\sigma_5$} (M-4-3);
  \draw[arr] (M-3-4) --node[above]{$\sigma_4$} (M-4-5);
  \draw[arr] (M-3-6) --node[above]{$\sigma_3$} (M-4-5);
  \draw[arr] (M-4-1) --node[above]{$\sigma_3$} (M-5-2);
  \draw[arr] (M-4-3) --node[above]{$\sigma_6$} (M-5-2);
\end{tikzpicture}
```

1.5



Note that even though the `\SkewTableau` commands are being used inside a `tikzpicture` environment they do not need coordinates because the skew tableaux are not explicit components in the environment, being all (implicitly) inside `TikZ`-nodes.

The following sections describe the commands defined by the `aTableau` package and how to use them. We start with the `\Tableau` command because all of the diagram and tableau commands, except for `\RibbonTableau`, are derived from `\Tableau`.

2. TABLEAUX AND YOUNG DIAGRAMS

Partitions, which are weakly decreasing sequences of non-negative integers, are fundamental objects in algebraic combinatorics and representation theory. Rather than working with sequences of integers, it is often useful to visualise partitions as Young *diagrams*, which are arrays of *boxes*, or *nodes*, in the plane³. A *tableau* is a (Young) diagram where the boxes are labelled using some alphabet. Equivalently, a diagram is a tableau with empty labels.

For example, $\lambda = (4, 3, 3, 2, 0, 0, \dots)$ is a partition of 12, since the entries sum to 12. It is customary to omit the zeros when writing partitions and to use exponents for repeated parts, so we can write this partition in *exponential notation* as $\lambda = (4, 3^2, 2)$. Many authors identify a partition $\lambda = (\lambda_1, \lambda_2, \dots)$ with its *diagram*, which is the set $[\lambda] = \{ (r, c) \mid 1 \leq c \leq \lambda_r \text{ for } r \geq 1 \}$. When using the *English convention*, the diagram of λ is visualised as a left justified array of boxes in the plane, where the first row has 4 boxes, the next two lower rows have 3 boxes and the last row has 2 boxes. The *shape* of a diagram is the partition that gives the number of boxes in each row. Diagrams and tableaux drawn using the *French convention* are given by reflecting in a horizontal line above the diagrams, whereas the *Ukrainian* and *Australian* conventions are given by appropriately rotating these diagrams. All of these conventions are used in the literature, with some papers using more than one convention.

2A. Tableaux. This section describes the `\Tableau` command, which draws tableaux or labelled diagrams. The syntax of this command is:

`\Tableau (x,y) [options] {tableau entries}`

(x,y) :
The Cartesian coordinates (x,y) are needed if and only if the tableau is a component of a `tikzpicture` environment.

options :
Optional arguments, described below.

tableau entries :
The `tableau entries` are given as a comma-separated list, with commas separating rows and each *token*, or braced-group of tokens, being in a separate column. As we explain below, each tableau entry can be prefixed by (optional) `TikZ`-styling specifications.

We show by example how to use the `\Tableau` command, starting with basic usage and finishing with style. Inside `\Tableau`, the rows are separated by commas, with the columns given by the “letters” in the word for each row:

1	2	3	4
5	6		
7			
8			

α	γ	δ	q
x	y	η	
λ	μ		
π	$\sum$		

```
\Tableau{ 1234, 56, 7, 8 }
\Tableau{ \alpha\gamma\delta q,
           xy\eta,
           \lambda\mu,
           \pi\sum }
```

2A.1

The tableau specifications can be put on a single line, without any spaces. Alternatively, as shown here, spaces and line breaks can be added to improve readability. The one restriction is that you cannot have blank lines inside a `\Tableau` command. As this example suggests, by default, the entries in a tableau are typeset in mathematics-mode, but this can be changed using the `text nodes` option, which is described below.

³In this manual, we normally refer to the entries of tableaux and diagrams as *boxes*, rather than *nodes*, to avoid any ambiguity with `TikZ`-nodes.

As the examples above show, the tableau entries can be individual characters, or they can be $\text{\TeX}$ commands. Tableaux frequently contain entries that are not single characters, or given by commands. Such entries can be added to a tableau by enclosing them in braces:

1	3	10	11	$2x$
2	x^2			

```
\Tableau{ 13{10}{11}{2x}, 2{x^2} }
```

2A.2

There is one additional caveat for braced-entries: any *singleton* braced entry `{...}` in a row needs to be *double-braced*. Double-bracing is only necessary when you have a row that has only one column, and you have a multi-letter column entry. The need for double-bracing is a consequence of how $\text{\LaTeX}$ 3 detects braced commas in comma-separated lists.

The following example shows the difference between single and double-bracing for tableau entries.

1	3	4	5
8	10		
1	1		
1	2		

1	3	4	5
8	10		
11			
12			

```
\Tableau{ 1345, 8{10}, {11}, {12} }
```

```
\Tableau{ 1345, 8{10}, {{11}}, {{12}} }
```

2A.3

The 10's in these tableaux do not need to be doubled-braced because the second row of these tableau has two columns. In contrast, because they are not doubled-braced, the 11 and 12 in the first tableau are both treated as being two characters in different columns in the first tableau. In the second tableau, the 11 and 12 both occupy a single column because they are double-braced.

2A.1. *Adding style.* The [aTableau](#) package provides several different ways to add [TikZ](#) style specifications to the entries in tableaux, and to boxes in diagrams. This is made possible by adding style prefixes to the tableau entries. To take advantage of these styles you will need at least rudimentary knowledge of how [TikZ](#)-styling works. The examples in this manual give a good indication of what is possible. For anything more exotic, please consult the [TikZ](#) manual.

The simplest way to change the style of an entry in a tableau is to add a `*`-prefix to its tableau specification.

1	2	3	4	5
6	7	8	9	
10	11	12		
13				

```
\Tableau{ 1*2*3*4*5, 6*789, {10}*{11}{12}, {{13}} }
```

2A.4

By default, the `*`-entries are given the [TikZ](#)-styling `fill=` . You can change the default `*`-style using the `star` style key:

1	2	3	4	5
6	7	8	9	
10	11	12		
13				

```
\Tableau[star style={fill=red!20,draw=red,thick}]
{ 1*2*3*4*5, 6*789, {10}*{11}{12}, {{13}} }
```

2A.5

Note that the `*`-styling overrides the default styling of the surrounding boxes. This is because the boxes with the default styling are placed first, in the order that they are entered, followed by boxes with custom styles are placed.

The `*`-syntax is a quick shorthand for adding emphasise some of the boxes in a tableau using a *single* style. It is possible to give every box in the tableau its own style by putting [TikZ](#)-styling specifications inside square brackets, `[...]`, before the corresponding letter in the tableau specification. You can mix the `*`-syntax and the `[...]`-style syntax for different entries, although only one of these style settings can be applied to any given box.

13				
10	11	12		
6	7	8	9	
1		3		

```
\Tableau[french]{
  1 [blue!20]2 [circle]3 [red]4 [cyan]5,
  6 *7 8 9,
  {10} *{11} {12},
  {{13}} }
```

2A.6

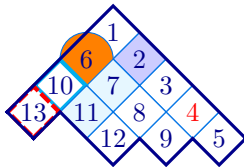
It is not necessary to add spaces between the entries in the way that this example does, which is done only to make it clearer how the style specifications are applied to the different boxes in the tableau.

Omitting some technicalities, each box in a tableau is constructed as a **TikZ** node, so any **TikZ**-styling specifications for a node can be used. This example shows that some care is needed when changing the style because simply giving a colour changes both the colour used to fill the box and the text colour, which is why some of tableau entries in the last example appear to be blank. If you are already familiar with **TikZ** then you probably already know what to do. If not, then here is a short list of useful style specifications for **TikZ** nodes:

Style	Meaning
<code>draw = <colour></code>	Sets the boundary colour of the node
<code>fill = <colour></code>	Sets the fill, or background, colour of the node
<code>font = </code>	Sets the font
<code>text = <colour></code>	Sets the text colour in the node

In the style settings, any valid L^AT_EX colour name can be used; see, for example, the **xcolor** manual. In addition, the style specifications `thin`, `thick`, `very thick`, `ultra thick`, `dashed`, `dotted`, ... change the border thickness, and its properties. Subsection 2A.3 below describes how to use these options to customise the tableau style.

If you want to apply “complicated” style settings, such as `draw=cyan,ultra thick` which consists of a comma-separated list of **TikZ**-style settings, then the entire style must be put inside matching square and curly braces `[{...}]`. This is necessary to ensure that L^AT_EX does not get confused by the commas in the style setting because commas are used to separate the rows of the tableau.



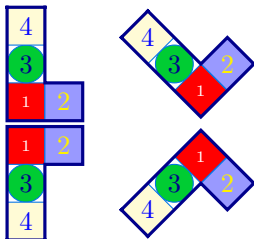
```
\Tableau[australian]{
  1 [fill=blue!20]2 3 [text=red]4 5,
  [{circle,fill=orange}]6 *7 8 9,
  [{draw=cyan,ultra thick}]{10} *{11} {12},
  [{dashed,draw=red,ultra thick}]{13} }
```

2A.7

The last example shows that it is not necessary to double-brace singleton entries when they have a style specification.

This example shows that modifying the borders of a box does adds only minimal emphasis. Use use this sparingly, especially since boxes placed later will redraw the border (see Subsection 2A.5). In addition, the orange circle in this example looks too big, but it is what we asked for because its diameter matches the horizontal width of the box. The next example shows that we get a better result for *Australian* and *Ukrainian* tableaux if we add a *minimum size* specification.

For complex or frequently used styles, we recommend using the `\tikzset` command to define the style outside of the `\Tableau` command. This makes the style both easier to read and easier to change.

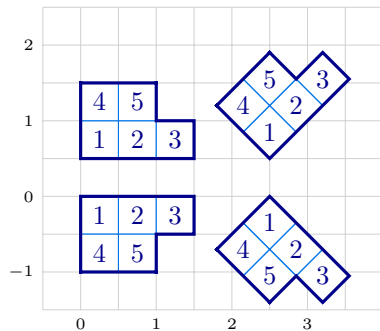


```
\tikzset{
  B/.style={fill=blue!40, text=yellow},
  G/.style={fill=green!80!blue,circle,minimum size=5mm},
  R/.style={fill=red,text=white, font=\tiny},
  Y/.style={fill=yellow!20, text=blue}
}
\Tableau[french]{[R]1[B]2,[G]3,[Y]4}
\Tableau[ukrainian]{[R]1[B]2,[G]3,[Y]4}
\Tableau[english]{[R]1[B]2,[G]3,[Y]4}
\Tableau[australian]{[R]1[B]2,[G]3,[Y]4}
```

2A.8

In this example, the **TikZ**-styles B, G R, and Y are applied to the tableaux entries labelled 1, 2, 3, and 4, respectively. Note the use of *minimum size* to make the circle the right size. The colours used here, and throughout this manual, are for demonstration purposes only: I recommend against such garish choices in any self-respecting document!

2A.2. *Tableau coordinates.* The (x,y) -coordinates are required if, and only if, `\Tableau` command is used inside a `tikzpicture` environment, in which case (x,y) gives the coordinates of the “outside corner” of the box in row 1 and column 1 of the tableau. The following example shows how tableaux are placed using (x,y) -coordinates inside a `tikzpicture` environment. The example also shows that, by default, the tableau boxes are half a unit wide.



```
\begin{tikzpicture}[add grid]
  \Tableau(0,0) [english] {123,45}
  \Tableau(0,0.5) [french] {123,45}
  \Tableau(2.5,0.5) [ukrainian] {123,45}
  \Tableau(2.5,0) [australian] {123,45}
\end{tikzpicture}
```

2A.9

2A.3. *Tableau options.* The optional $*$ -styles and $[...]$ -styles are the main mechanism for styling individual nodes in a tableau. Using a key-value syntax, you can change aspects of the tableaux produced by the `\Tableau` command. The rest of this section describes these keys, their default values, and gives examples of how to use them.

The options below can be applied to the `\Tableau` command by giving them as a comma-separated list inside square brackets:

`\Tableau [options] {tableau entries}` or `\Tableau (x,y) [options] {tableau entries}`.

The options can appear in any order, with later options having precedence over earlier ones.

When used inside a `\Tableau` command, the options only affect that particular tableau. Most of these options, or settings, can also be used in the `\aTabset` command, or as optional arguments in `\usepackage[...]{atableau}`, to change the default settings of all tableaux in the same L^AT_EX group. For example, if you put the command `\aTabset[ukrainian]` in the preamble of your document then, by default, all subsequent tableaux and Young diagrams will be drawn using the Ukrainian convention.

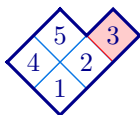
english (default: **english**)

french

ukrainian

australian

These four (mutually exclusive) options change the convention, or orientation, that is used when drawing the tableaux. Rather than define this precisely, we use the tired and true method in combinatorics of defining by example.



```
\tikzset{R/.style={fill=red!20,draw=red}}
\Tableau[ukrainian]{12[R]3,45}
```

2A.10



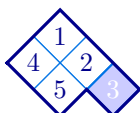
```
\tikzset{R/.style={fill=red!20, circle, draw=red, ultra
thick}}
\Tableau[french]{12[R]3,45}
```

2A.11



```
\tikzset{B/.style={fill=blue!20, dashed, thick}}
\Tableau[english]{12[B]3,45}
```

2A.12



```
\tikzset{B/.style={fill=blue!20, thick, text=white}}
\Tableau[australian]{12[B]3,45}
```

2A.13

By default, the english convention is used to draw tableaux and Young diagrams. The default convention can be changed using `\aTabset`. Capitalised names, `Australian`, `English`, `French` and `Ukrainian`, are also recognised. If, for example, your document is only going to draw french tableaux and diagrams, then you can specify this when you load the package using `\usepackage[french]{atableau}`, or by adding `\aTabset{french}` to your document preamble.

align (default: `centre`)

[accepts: `centre`/`bottom`/`top`]

Use the `align` key to change the vertical centering of tableaux. By default, tableaux are aligned on their baseline.

1	2	3
4	5	5
6	7	4

1	2	3
4	5	5

```
\Tableau{123,455,674}
\Tableau{123,455}
```

2A.14

1	2	3
4	5	5
6	7	4

1	2	3
4	5	5

```
\Tableau[align=top]{123,455,674}
\Tableau{123,455}
```

2A.15

1	2	3
4	5	5
6	7	4

1	2	3
4	5	5

```
\aTabset{align=bottom}
\Tableau{123,455,674}
\Tableau{123,455}
```

2A.16

1	2	3
4	5	5
6	7	4

1	2	3
4	5	5

```
\aTabset{align=top}
\Tableau{123,455,674}
\Tableau{123,455}
```

2A.17

It is not necessary to have the same `align` keys on each tableau.

border (default: `true`)

[accepts: `true`/`false`]

no border (default: `false`)

[accepts: `false`/`true`]

The `border` and `no border` options enable and disable the drawing of the border wall around a tableau. As with other boolean keys, `border` is equivalent to `border=true`, and `no border` is equivalent to `no border=true`. Further, using `no border` is equivalent to `border=false` and, similarly, `border` is equivalent to `no border=false`.

	4	
7		5
6	5	3
	4	2
	1	

6	7	4
4	5	5
1	2	3

```
\Tableau[ukrainian, border=false]{123,455,674}
\Tableau[french, no border]{123,455,674}
```

2A.18

border colour (default: `black`)

[accepts: any colour]

border style (default: `thick,line cap=rect`)

[accepts: `TikZ`-styling]

The `border colour` option sets the colour of the border wall, whereas `border style` sets the `TikZ`-styling of the border wall. (So, `border style` can override `border colour`.)

	4	
7		5
6	5	3
	4	2
	1	

4	5	5
1	2	3

```
\Tableau[ukrainian, border colour=red]{123,455,674}
\Tableau[french, border style=dashed]{123,455}
```

2A.19

The `border style` appends any `TikZ`-styles to the current styling of the wall.

box height (default: `0.5`)

[accepts: a decimal giving the height in cm]

box width (default: `0.5`)

[accepts: a decimal giving the width in cm]

aTableau

Sets the width and height of the boxes in the tableau. The default height and width of the boxes is 0.5 cm when using the english and french conventions and 0.7012 cm, which is approximately $1/\sqrt{2}$ cm, when using the ukrainian and australian conventions. (In all cases, the side length of the squares is 0.5 cm.)

1	2	3
4	5	5
6	7	4

1	2	3
4	5	5

```
\aTabset{align=top}
\Tableau[box height=0.7]{123,455,674}
\Tableau[box width=0.3]{123,455}
```

2A.20

See also the `scale`, `xscale` and `yscale` options.

conjugate (default: `false`)

[accepts: `false/true`]

Draws the *conjugate* tableau, where the rows and columns are swapped.

1	2	3
4	5	
6	7	

1	4	6
2	5	7
3		

```
\Tableau{123,45,67}
\Tableau[conjugate]{123,45,67}
```

2A.21

inner colour (default: `blue`)

[accepts: any colour]

inner style (default: `thin,line cap=rect`)

[accepts: `TikZ`-styling]

The inner colour option sets the colour of the inner wall, whereas inner style sets the `TikZ`-styling of the inner wall. (So, inner style can override inner colour.)

1		2
4		3
6	5	4
7	5	
	4	

1	2	3
4	5	5

```
\Tableau[australian, inner colour=red]{123,455,674}
\Tableau[inner style=dashed]{123,455}
```

2A.22

To remove the inner tableau walls use `inner colour=none`.

7		3
6	5	
4	2	
	1	

```
\Tableau[ukrainian, inner colour=none]{123,45,67}
```

2A.23

box fill (default: `none`)

[accepts: any colour]

The `box fill` option sets the background colour of a box in a tableau, just like the `fill` key for a `TikZ`-node. If you use a dark fill colour, then you will almost certainly want to change the text colour using the `text` option – and you may also want to change the colours of the border walls and inner walls using `border colour` and `inner colour`, respectively.

1	2	3
4	5	5
6	7	4

1	2	3
4	5	5

```
\Tableau[box fill=blue,box text=yellow]{123,455,674}
\Tableau[box fill=red!10]{123,455}
```

2A.24

box font

[accepts:]

The `box font` key sets the font used to type the entries of a tableau. By default, tableau entries are typeset as mathematics, so this is mainly useful for changing the font size (because, for example `\bfseries $1$` does not make the 1 bold). It is only when you are using text nodes that font commands like `\itshape` and `\bfseries` will have any effect.

1	2	3
4	5	5
6	7	4

1	2	3
4	5	5

```
\Tableau[box font=\tiny]{123,455,674}
\Tableau[text nodes, box font=\itshape]{123,455}
```

2A.25

box text (default:)

[accepts: any colour]

Exactly as for a **TikZ**-node, use **text** to set the default text colour for the tableau entries.

1	2	3
4	5	

4	5	
1	2	3

```
\Tableau[box text=red]{123,45}
\Tableau[box text=blue, french]{123,45}
```

2A.26

math nodes (default: **true**)

[accepts: true/false]

text nodes (default: **false**)

[accepts: true/false]

Use the **math nodes** and **text nodes** to have the tableau entries typeset as either mathematics or text, respectively. By default, the entries of a tableau are typeset as mathematics.

A	B	C	D
e	f	g	
H	I		

A	B	C	D
e	f	g	
H	I		

```
\Tableau[math nodes]{ABCD, efg, HI}
\Tableau[text nodes]{ABCD, efg, HI}
```

% the default

2A.27

name (default: **B**)

[accepts: string/character]

By default, the boxes can be referenced using the node names (B-1-1), (B-1-2), (B-1-3), ..., with (B-*r*-*c*) referring to the node in row *r* and column *c*. This feature is most useful when you have several tableaux inside a **tikzpicture** environment because if you only have one tableau, then it is unlikely that you need to change the default prefix for the node names. Examples are given in ?? and ??.

See the introduction to this section for examples and for more information about using node names and anchors.

ribbons[accepts: list of ribbon specifiers] **snoobs**

[accepts: list of ribbon specifiers]

Use the **ribbons** key to add a comma-separated list of ribbons to a tableau. Use **snoobs** to add ribbons to the tableau that are placed *after* the border of the diagram is drawn. (Writing *ribbons* backwards gives *snobbir*.)

When adding ribbons, or snoobs, you first specify the row and column indices of the *head* of the ribbon, which is the unique node of maximal content in the ribbon, and then give a sequence of *r*'s and *c*'s depending on whether the ribbon moves along a row or column. As always, you have the option of adding style — and you can also specify the contents of each box in the ribbon. For more details about the ribbon specifications, with examples, see [Section 2F](#), which describes the **\RibbonTableau** command.

Use the **ribbon style** option to change the default style of the ribbons. For example, noting that a box is a ribbon of length 1, the following code uses ribbons of length one (that is, nodes), to highlight the addable nodes of this tableau.

1	2	3	4	A
5	6	7	B	
8	9	10		
11	C			
D				

```
\Tableau[star style={fill=green!80!blue,text=white},
ribbon box={fill=yellow,text=red},
ribbons={15_A,24_B,42_C,51_D},
]{123*4,567,89*{10},*{11}}
```

2A.28

As this example indicates, ribbons are assumed to be contained inside the diagram of the tableau, so they do not change the border of a tableau.

The default style for a **snoobs** ribbon is given by **ribbon style**.

1	2	3	4	A
5	6	7	B	
8	9	10		
11	C			
D				

```
\Tableau[star style={fill=green!80!blue,text=white},
ribbon box={fill=yellow, text=red},
snoobs={15_A,24_B,42_C,51_D},
]{123*4,567,89*{10},*{11}}
```

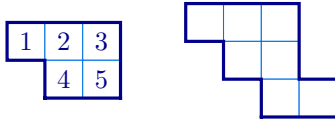
2A.29

The difference between these two pictures is that in the second picture the nodes in the **sno**bs are drawn over the tableau border, causing the border to disappear for these nodes. For this reason, in most cases you should use ribbons, and avoid **sno**bs.

shifted (default: **false**)

[accepts: true/false]

Set to **true** for a shifted tableau or shifted diagram. This is discussed in more detail in [Section 2D](#), which describes the `\ShiftedDiagram` and `\ShiftedTableau` commands.



```
\Tableau[shifted]{123,45}
\Diagram[shifted]{3,2^2}
```

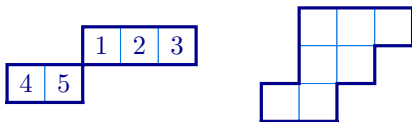
2A.30

See [Section 2D](#) for the options specific to shifted tableaux and shifted diagrams.

skew (default: **(0)**)

[accepts: a partition]

Specify the skew shape for a skew tableau or skew diagram. This is discussed in more detail in [Section 2C](#), which describes the `\SkewDiagram` and `\SkewTableau` commands.



```
\Tableau[skew={2}]{123,45}
\Diagram[skew={2^2,1}]{3,2^2}
```

2A.31

See [Section 2C](#) for the options specific to skew tableaux and skew diagrams.

star style (default: **fill=** )

[accepts: **TikZ**-styling]

Use the **star style** to add to the style of the starred entries of a tableau. (In **TikZ** parlance, **star style** appends appends these styles to the default **star style**.)



```
\Tableau[star style={text=magenta,
draw=red, thick, dashed}]{1*2*3,4*5}
\Tableau[star style={fill=orange}]{1*2*3,4*5}
```

2A.32

Note that the tableau border is drawn *after* the **star style** is applied. You can use **sno**bs to draw on top of the border.

scale (default: **1**)

[accepts: a decimal number]

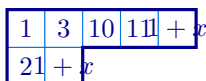
xscale (default: **1**)

[accepts: a decimal number]

yscale (default: **1**)

[accepts: a decimal number]

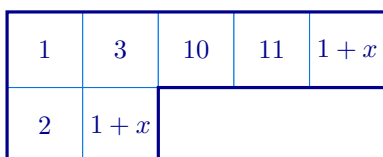
By design, boxes in tableaux do not automatically resize to fit their contents. For example, consider:



```
\Tableau{ 13{10}{11}{1+x}, 2{1+x} }
```

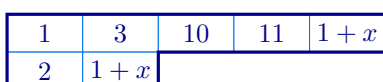
2A.33

This is unlikely to be the desired output! The options **scale**, **xscale** and **yscale** can be used to rescale the boxes in tableaux and diagrams. The names are slightly misleading because **xscale** rescales in the direction of increasing column index and **yscale** rescales in the direction of increasing row index.



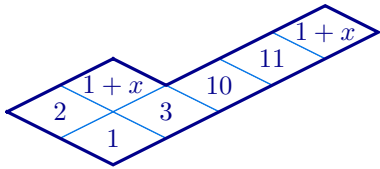
```
\Tableau[scale=2]{ 13{10}{11}{1+x}, 2{1+x} }
```

2A.34



```
\Tableau[xscale=2]{ 13{10}{11}{1+x}, 2{1+x} }
```

2A.35



```
\Tableau[ukrainian, xscale=2]{ 13{10}{11}{1+x}, 2{1+x} }
```

2A.36

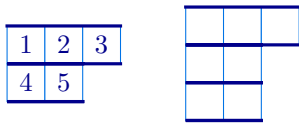


The options, scale, xscale, and yscale, affect all `aTableau` diagrams, including tableaux, Young diagrams and abacuses. If you want to use `\aTabset` to change the sizes of all diagrams and tableaux in your document it is better to use the box height and box width options.

`tabloid` (default: `false`)

[accepts: `true/false`]

Set to `true` for a tabloid. This is discussed in more detail in [Section 2E](#), which describes the `\Tabloid` command.



```
\Tableau[tabloid]{123,45}
\Diagram[tabloid]{3,2^2}
```

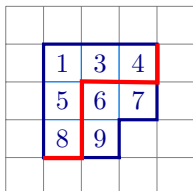
2A.37

See [Section 2E](#) for the options specific to skew tabloids.

`tikz before`
`tikz after`

[accepts: `TikZ` commands]
[accepts: `TikZ` commands]

These keys inject `TikZ` code into the underlying `tikzpicture` environment, where the tableau is constructed, before and after the `\Tableau` command, respectively.



```
\Tableau[
  tikz before={\draw[help lines,step=0.5](-0.5,-2)grid(2,0.5)};},
  tikz after={\draw[ultra thick,red]
    (B-1-3.north east)--(B-1-3.south east)
    --(B-2-1.north east)--(B-3-1.south east)
    --(B-3-1.south west);
  }]{134,567,89}
```

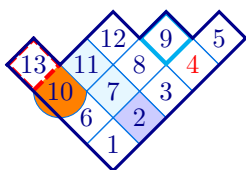
2A.38

Both before and after hooks are provided because the order in which you place `TikZ` commands can affect the drawing, primarily because later commands are drawn over the top of earlier ones. Note that the *before* code cannot use the named nodes `B-r-c` because this code is executed before the tableau is drawn, which is when these nodes are constructed. For examples, see [Subsection 2A.5](#).

`tikzpicture`

[accepts: `TikZ`-styling]

This command adds an optional argument to the underlying `tikzpicture` environment, which contains the tableau. The `tikzpicture` option is ignored when the `\Tableau` command is equipped with (x,y) -coordinates.



```
\Tableau[ukrainian, name=A,
  tikzpicture={remember picture}}
{ 1[fill=blue!20]23[text=red]45,
  6*78[{\draw=cyan,ultra thick}]9,
  [{\circle,fill=orange}]{10}*{11}{12},
  [{\dashed,draw=red,ultra thick}]{13}
}
```

2A.39

Since `name=A`, the nodes in this tableau are referenced as (A-1-1), (A-1-2), (A-1-3), As the example uses *remember picture*, other `tikzpicture` environments can reference the node names in last tableau, which allows us to do things like this:

This circle is too big!
Use `minimum size=5mm`

```
\tikz[remember picture, overlay]
\draw[very thick, ->,red]
(0,0) node[right,align=left]
{This circle is too big!\Use minimum size=5mm}
to [out=135, in=225] (A-3-1);
```

As a second example, we can use `tikzpicture={rotate=30}` to try to rotate a tableau by 90 degrees with:

3	
2	5
1	4

```
\Tableau[tikzpicture={rotate=90}]{123,45}
\begin{tikzpicture}[rotate=90]
\Tableau(0,0){123,45}
\end{tikzpicture}
```

The two tableaux constructions in this example are equivalent. An oddity with both of these examples is that the numbers in the tableaux have not been rotated, which is because the `TikZ rotate` key does not affect component objects, such as nodes, inside a `TikZ` picture. You can rotate everything in the picture using the *transform canvas* key:

3	
2	5
1	4

```
\Tableau[tikzpicture={transform canvas={rotate=90}}]{123,45}
```

2A.4. *Compositions*. As we have defined them, tableaux are always of *partition* shape, in the sense that the lengths of the rows is a weakly decreasing sequence. In fact, the `\Tableau` command also accepts tableaux of *composition* shape, where the lengths of the rows are not necessarily in decreasing order

4	5		
2	7	5	1
3	1		

```
\Tableau{ 45,2751,31 }
\SkewTableau{1,2}{ 45,27,31 }
```

Tableaux and diagrams should not contain empty rows, and the diagrams should be connected. (For disconnected diagrams, see [Section 2G](#).)

2A.5. *Drawing order*. When `TikZ` constructs a picture, later objects are drawn over the top of earlier ones, sometimes obscuring earlier features. For this reason it is useful to know the drawing order used by :

- First, the tableau entries that use the default styling are placed, in the order that appear in the *tableau specifications*.
- Secondly, the styled tableau entries are placed, in the order that they appear in the *tableau specifications*.
- Next, the nodes in any ribbons are placed, again in the order that the ribbons are listed. For each ribbon, first the border of the ribbon is drawn, together with any styling for the ribbon, and then the boxes in the ribbon, together with any styling, are added.
- When enabled by `skew border`, the skew border is drawn, after which the border of the tableau is drawn.
- Finally, any ribbons specified using `snoobs` are drawn, in the order that they are listed.

2A.6. *Special characters*. The characters `[]`, `,`, and `*` have special meanings in the *tableau specifications*. If you want these characters to be entries in a tableau, then you need to enclose them in braces:

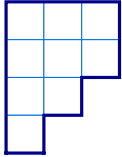
1	3	,	5	]
8	[			
*				

```
\Tableau{ 13{,}5], 8{[}, {{*}} }
```

2A.44

There is no need to enclose `]` in braces because it only becomes special when it is preceded by a `[` character.

2B. Young diagrams. A *Young diagram*, or simply a *diagram*, is an unlabelled tableau. Diagrams can be drawn using the `\Tableau` command:



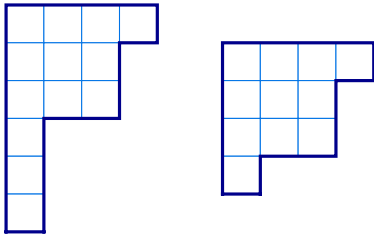
```
\Tableau{~~~,~~~,~~,~}
```

2B.1

This approach works but it is slightly cumbersome and hard to proofread, so `aTableau` provides the `\Diagram` command. The `\Diagram` command uses almost the same syntax as the `\Tableau` command:

```
\Diagram (x,y) [options] {partition}
```

As with the `\Tableau` command, the (x,y) -coordinates are needed if and only if the diagram is inside a `tikzpicture` environment. The `\Diagram` command allows partitions to be specified either as a comma-separated list of weakly decreasing nonnegative integers, or in *exponential notation*:



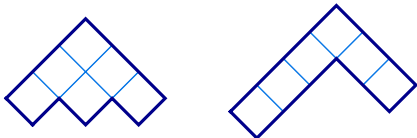
```
\aTabset{align=centre}  
\Diagram{4,3^2,1^3}  
\Diagram{4,3,3,1}
```

2B.2

Internally, `\Diagram` actually does something like the first example in this section, so all of the options for the `\Tableau` command can be used with `\Diagram`. We do not repeat all of these options in this section and, instead, highlight some of the more interesting ones together with new options that apply specifically to diagrams.

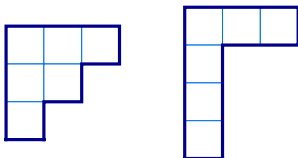
Just like the `\Tableau` command, `\Diagram` supports the four different conventions for diagrams, with `english` being the default:

`english`
`french`
`ukrainian`
`australian` (default: `english`)



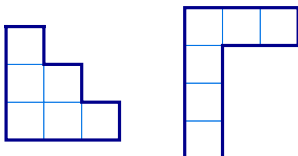
```
\Diagram[australian]{3,2,1}  
\Diagram[australian]{3,1^3}
```

2B.3



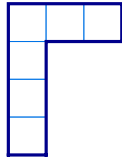
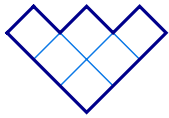
```
% English is the default convention  
\Diagram[english]{3,2,1}  
\Diagram{3,1^3}
```

2B.4



```
\Diagram[french]{3,2,1}  
\Diagram{3,1^3}
```

2B.5



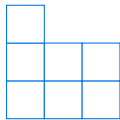
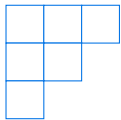
```
\Diagram[ukrainian]{3,2,1}
\Diagram{3,1^3}
```

2B.6

border (default: **true**)
no border (default: **false**)

[accepts: true/false]
 [accepts: true/false]

If **border** is true, then the border of the tableau is drawn. If **border** is false, then the border of the tableau is not drawn. By default, the border is drawn. The **no border** option, which is provided only for completeness, is the inverse of **border**. That is, **no border** is equivalent to **border=false**, and **no border=false** is equivalent to **border=true**.



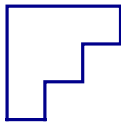
```
\Diagram[border=false]{3,2,1}
\Diagram[no border, french]{3^2,1}
```

2B.7

border only (default: **false**)

[accepts: false/true]

When **border only** is set to true, only the border of the tableau is drawn, and not the internal walls around the boxes.



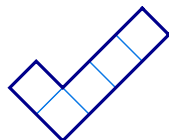
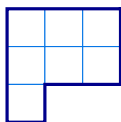
```
\Diagram[border only]{3,2,1}
```

2B.8

conjugate (default: **false**)

[accepts: false/true]

Draws the conjugate diagram, which has the rows and columns interchanged.



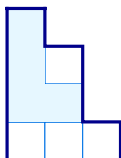
```
\Diagram[conjugate]{3,2,2}
\Diagram[ukrainian, conjugate]{2,1^3}
```

2B.9

ribbons
snobs

[accepts: ribbon specification]
 [accepts: ribbon specification]

Use **ribbons** and **snobs** to add ribbons to a diagram.

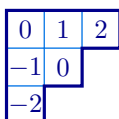


```
\Diagram[french, ribbons={*22*c*r*r}]{3,2^2,1}
```

2B.10

show [accepts: contents,final,hooks,initial,residue] Use **show** option to draw particular types of tableau of the specified shape. The possible choices are:

- **show=contents** print the diagram where each box is labelled by its *content*, which is the row index minus the column index.



```
\Diagram[show=contents]{3,2,1}
\Diagram[french,show=contents]{2^2,1}
```

2B.11

- `show=final` print the *final tableau* of this shape, which has the numbers $1, 2, \dots, n$ entered in order from top to bottom down successive columns (with appropriate modifications for the non-English conventions).

1	4	7
2	5	
3	6	

	1	
2		4
3	5	

```
\Diagram[show=final]{3,2,2}
\Diagram[australian,show=final]{2^2,1}
```

2B.12

- `show=hooks` print the diagram where each box is labelled by the corresponding *hook length*. If λ is a partition and λ' is its conjugate partition then the hook length of the box in row i and column j is $\lambda_i - i + \lambda'_j - j + 1$.

5	4	1
3	2	
2	1	

1	1	
3		2
	4	

```
\Diagram[show=hooks]{3,2,2}
\Diagram[ukrainian,show=hooks]{2^2,1}
```

2B.13

- `show=initial` print the *initial tableau* of this shape, which has the numbers $1, 2, \dots, n$ entered in order from top left to right along down successive rows (with appropriate modifications for the non-English conventions).

1	2	3
4	5	
6	7	

7	8	6
	5	3
4	2	
	1	

```
\Diagram[show=initial]{3,2,2}
\Diagram[ukrainian,show=initial]{3^2,2}
```

2B.14

- `show=residues` print the diagram where each box is labelled by its *residue*, which is the row index minus the column index modulo some integer e , which must also be supplied.

0	1	0
1	0	
0		

1		
2	0	
0	1	2

```
\Diagram[show=residues, e=2]{3,2,1}
\Diagram[french,show=residues, e=3]{3,2,1}
```

2B.15

2C. Skew tableaux and skew diagrams. A partition μ is *contained* in another partition λ , which is written as $\mu \subseteq \lambda$, if $\mu_k \leq \lambda_k$, for $k \geq 0$. If $\mu \subseteq \lambda$ then the *skew partition* $\lambda/\mu = \{(r, c) \in \lambda \mid (r, c) \notin \mu\}$ is the set of nodes that are in λ and not in μ . In this manual, μ is the *inner shape* and λ/μ is the outer shape. A *skew tableau* is a labelling of the nodes in the diagram of a skew partition. Skew partitions and skew tableau can be drawn using the commands:

```
\SkewDiagram (x,y) [options] {inner shape} {outer shape}
\SkewTableau (x,y) [options] {inner shape} {skew tableau}
```

As with the previous commands, the (x, y) -coordinates should be given if and only if the picture is inside a `tikzpicture` environment. The inner and outer shapes can either be given as a comma-separated list of integers or in exponential notation.

In fact, the `\SkewTableau` is just an alias for the `\Tableau`, using the `skew` key to set the inner shape. For this reason, all of the options for the `\Tableau` command can be used with `\SkewTableau`. In particular, the entries in a `\SkewTableau` are specified in exactly the same way as in the `\Tableau` command. In particular, the entries of skew tableaux can, optionally, be given style prefixes, both using a `*` to add the current `star` style, or arbitrary `TikZ`-styling using `[...]`.

7	9	
	5	7
	1	3

7	9	
	5	7
	1	3

```
\tikzset{R/.style={fill=red!20,text=red}}
\SkewTableau[french]{1^2}{13,5*7, [R]79}
\Tableau[french,skew={1^2}]{13,5*7, [R]79}
```

2C.1

Similarly, the `\SkewDiagram` command is an alias for the `\Diagram` command, with a `skew` key.


```
\SkewDiagram[french]{1^2}{2^3}
\Diagram[french,skew={1^2}]{2^3}
```

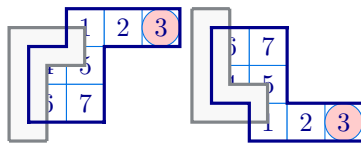
2C.2

Since they are just aliases, all of the options for the `\Tableau` and `\diagrams` commands can be used with the `\SkewTableau` and `\SkewDiagram` commands. In addition, the following options are supported:

Skew tableau and skew diagrams should always be connected. Use `\Multidiagrams` and `\Multitableau` to draw disconnected diagrams.

skew border (default: `false`) [accepts: `true/false`]
no skew border (default: `true`) [accepts: `false/true`]

When skew border is true, the border of the (inner) skew shape is drawn.



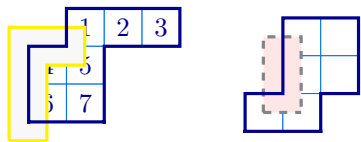
```
\tikzset{R/.style={fill=red!20,circle}}
\SkewTableau[skew border]{2,1^2}{12[R]3,45,67}
\SkewTableau[french,skew border]{2,1^2}{12[R]3,45,67}
```

2C.3

Note that skew border only draws the border of the skew shape and not the boxes inside the inner skew shape. To draw the boxes inside the skew shape use skew boxes

skew border style (default: `draw=` , `fill=` , `very thick`) [accepts: `TikZ`-styling]

Use skew border style to change the style of the skew border. Since the skew border is only drawn then skew border is set, the skew border style does not have any effect unless skew border has been set to true.



```
\SkewTableau[skew border style={draw=yellow},
skew border]{2,1^2}{123,45,67}
\SkewDiagram[skew border style={dashed,fill=red!10},
skew border]{1^2}{2^3}
```

2C.4

To do: Implement the following options

skew boxes (default: `false`) [accepts: `true/false`]

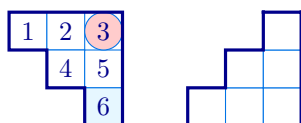
Draw the inner walls in the inside the inner partition of a skew

skew boxes style (default: `thin,gray`) [accepts: `TikZ`-styling]

2D. Shifted tableau and shifted diagrams. *Strict partitions*, which have strictly increasing parts, and *shifted tableaux*, which are of strict partition shape, appear in several places in representation theory, such as the projective representations of the symmetric groups. These tableaux and diagrams are drawn with row r shifted by r -units along the row, so that the first box in each row has content 0. These diagrams and tableaux can be drawn using the following commands:

```
\ShiftedDiagram (x,y) [options] {partition }
\ShiftedTableau (x,y) [options] {tableau specification}
```

As usual, the (x, y) -coordinates are necessary if and only if these commands are used inside a `tikzpicture` environment. The partition in a `\ShiftedDiagram` can be given using exponential notation and the tableau specification for a `\ShiftedTableau` can include the usual optional style prefixes.



```
\tikzset{R/.style={fill=red!20,circle}}
\ShiftedTableau{12[R]3,45,*6}
\ShiftedDiagram[french]{3,2,1}
```

2D.1

In the literature, shifted tableaux and shifted diagrams almost always have strict partition shape, however, this is not enforced by these commands. Under the hood, the `\ShiftedTableau` is the `\Tableau` command with the `shifted` option set to true. Similarly, `\ShiftedDiagram` is the same as using the `\Diagram` with the `shifted` option. Consequently, all of the options for the `\Tableau` and `\Diagram`